Kru — Project Plan & Milestones

Nov 1, 2026 · @Amin Haiqal

Overview

Kru is a headless, local-first engine that runs AI agent CLIs (Claude Code, Codex, Gemini, Cursor and others) as persistent teammates. Every agent gets a durable mailbox, a workspace and a thread that survives provider switches and crashes. Work is delivered one turn at a time, and agents message each other through the same mailbox.

Pitch: *Give every agent a mailbox. Kru delivers the work and never loses a message.*

The name: *kru* is Malay for "crew" — a crew of agents that hand work to each other. One syllable, easy to say, easy to type as a command.

Who uses it

- Developers who want several agent CLIs to work on one codebase, each with its own workspace and memory.
- Teams that put agents behind their own tools: a Slack bot, a CI job, an internal dashboard.
- Builders who need an engine they can embed, not a desktop app.

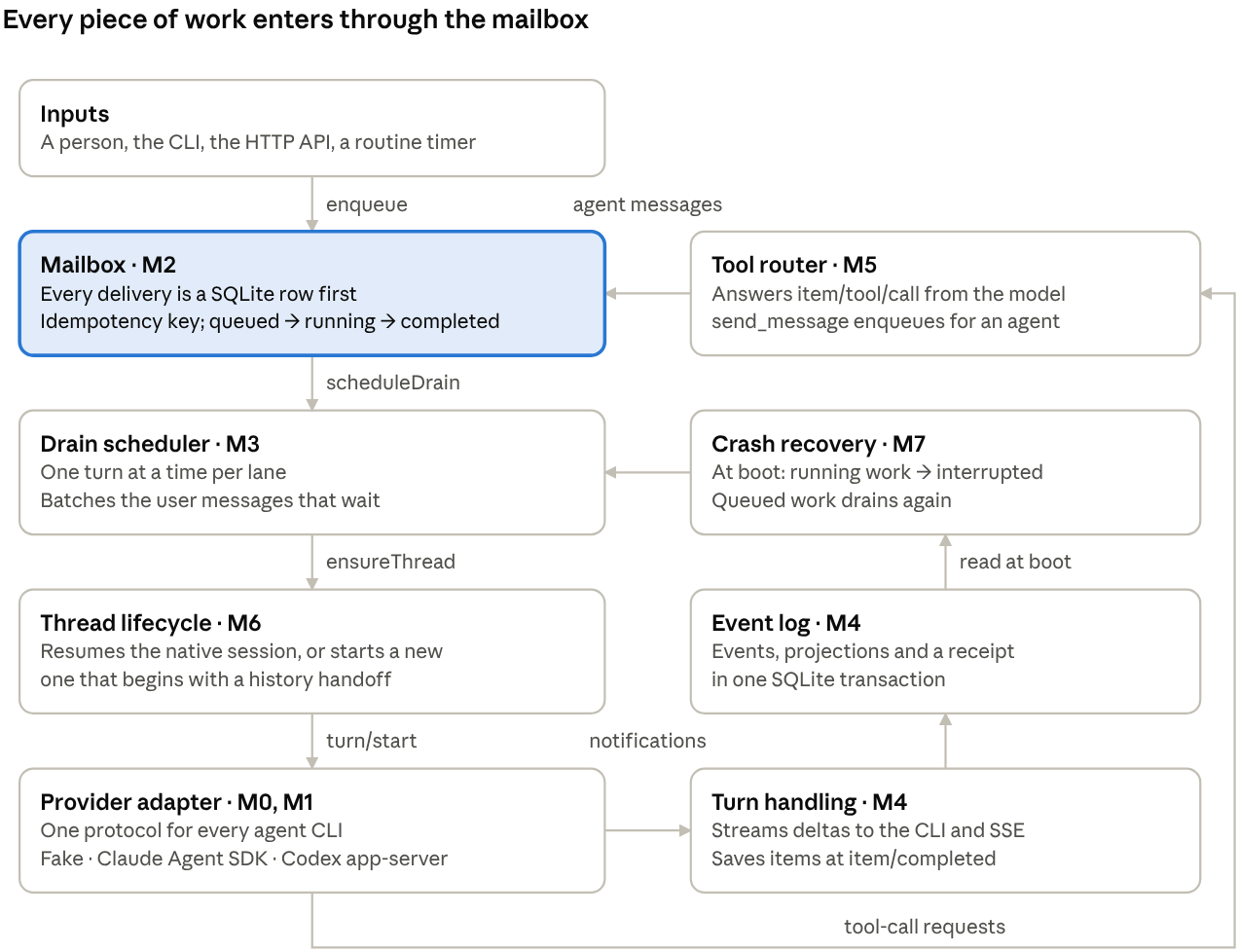
How it differs from OpenBot

	OpenBot	Kru
Form	Electron desktop app	Daemon, CLI, HTTP + SSE API, TypeScript library
Audience	End users	Developers and integrators
Scope	Browser, mobile, voice, team servers	The core engine only
License	PolyForm Noncommercial 1.0.0	MIT (allows commercial use)
Code	—	Written from scratch from the ideas; no copied code

The license row is why the last row matters: OpenBot's code cannot go into an MIT project, so study a file, close it, then write your own version.

Architecture

Work flows down the left column and results flow back up the right. Agent messages loop back into the same mailbox, so there is only one execution path.



Kru message lifecycle · 9 parts, labelled by milestone

Core concepts

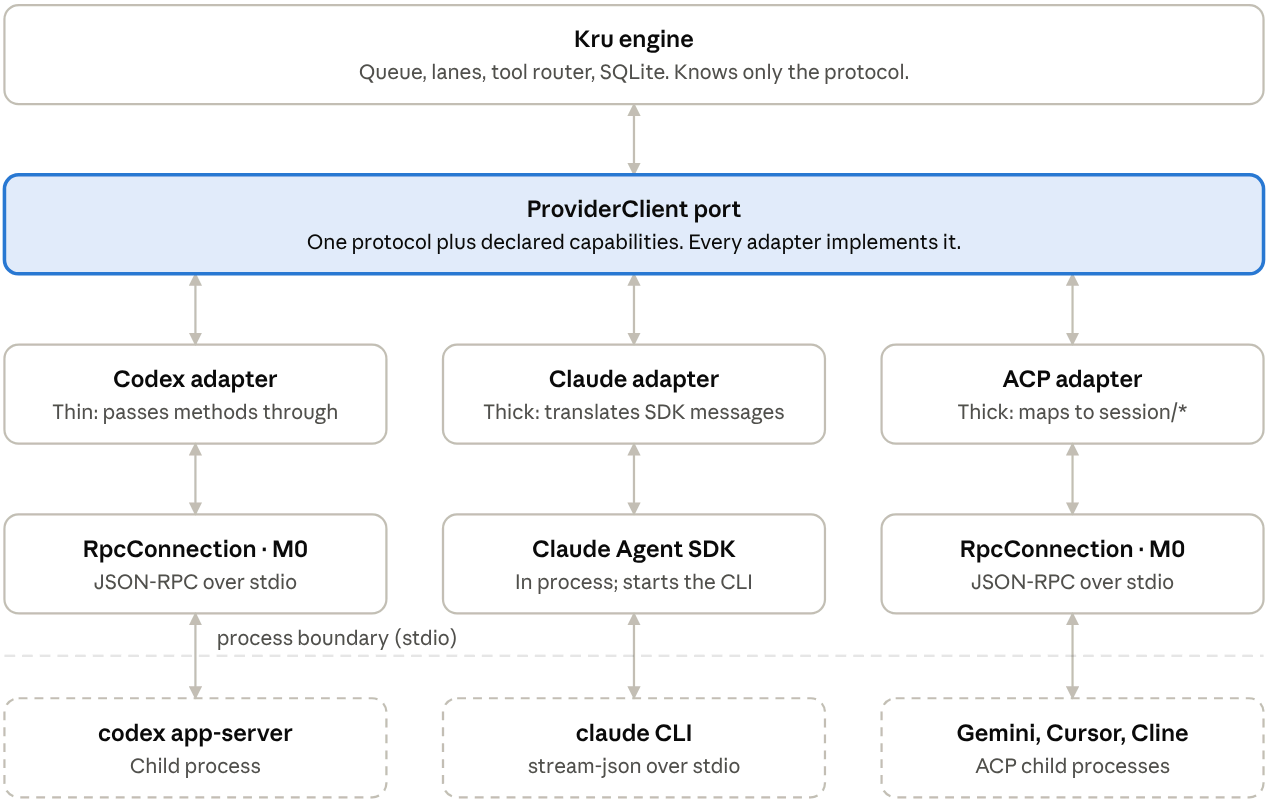
- **Delivery:** one unit of work for one agent, stored before anything runs.
- **Lane:** a queue that runs one turn at a time — an agent's main chat, or one of its side threads.
- **Thread vs provider session:** the thread is Kru's and permanent; the session is the CLI's and disposable.
- **Handoff:** a size-limited summary of recent history, prepended to the first turn of a new session.

- **Turn:** one `turn/start` to `turn/completed` exchange with a provider.

Provider layers: ports and adapters

Kru's engine depends on one interface, never on a specific agent. This is hexagonal architecture, also called ports and adapters: the `ProviderClient` interface is the port, and each agent CLI plugs in through its own adapter. Adding an agent means adding one adapter; the engine does not change.

The engine talks to one port; adapters absorb each agent difference



Kru provider layers · engine, port, 3 adapters, transports, agent processes

Arrows run both ways: commands go down, events and tool-call requests come up. Dashed boxes are separate processes that Kru starts and can lose at any time.

What each layer owns

Layer	Owns	Knows about	Milestone
Engine	What runs and when: queue, lanes, tool router, SQLite	The protocol only	M2–M7

Layer	Owns	Knows about	Milestone
`ProviderClient` port	The contract: protocol methods and declared capabilities	Nothing below it	M1
Adapters	Translation between the protocol and one agent's native API	One agent	M1
Transport	Framed messages over stdio, request ids, timeouts	Bytes and JSON-RPC	M0
Agent process	The model, its tools, its own session files	Nothing about Kru	External

The shared language. OpenBot uses the Codex App Server protocol as the language every adapter speaks, because Codex already had a clean JSON-RPC design. Kru keeps the same method names (see Core types) but gives each method's params and result a TypeScript type, so the compiler checks every adapter. JSON-RPC framing stays inside the transport.

How each adapter translates

Kru protocol	Codex adapter	Claude adapter	ACP adapter
`thread/start`	Sent as-is	Creates an SDK session itself	`session/new`
`turn/start`	Sent as-is	Calls the SDK's `query()` with the prompt	`session/prompt`
`item/agentMessage/delta` (up)	Received as-is	Built from SDK assistant messages	Built from `session/update` (`agent_message_chunk`)
`turn/completed` (up)	Received as-is	Built when the SDK result message arrives	Built when `session/prompt` returns

Kru protocol	Codex adapter	Claude adapter	ACP adapter
`item/tool/call` (up, request)	Received as-is	Built from the SDK tool callback	Built from ACP tool requests

The Codex adapter is thin. The Claude and ACP adapters are thick — about 2,400 and 3,150 lines in OpenBot — because they act as a JSON-RPC server: they answer requests themselves and build notifications from what their agent emits.

One turn through the layers (Claude)

1. The engine calls `turn/start` with the thread id and the prompt.
2. The Claude adapter calls `query()` and answers at once with the new turn id.
3. The SDK writes to the `claude` CLI's stdin and reads its stdout as stream-json.
4. Text arrives; the adapter emits `item/agentMessage/delta` notifications up to the engine.
5. The model calls `send_message`; the adapter sends an `item/tool/call` request up and waits for the engine's answer.
6. The SDK reports the result; the adapter emits `turn/completed`.

The engine sees the same six events it would see from Codex.

Where state lives

Layer	Keeps	When it is lost
Transport	Pending requests by id	Rejected on process exit (M0)
Adapter	Native session ids, the active turn	Session resumed or replaced with a handoff (M6)
Engine	Threads, deliveries, messages in SQLite	Never: SQLite is the source of truth

Rules that keep the layers clean

1. Dependencies point inward: the engine imports the port, never an adapter. Adapters are chosen in one registry.
2. An adapter never writes to SQLite or to the screen. It only emits protocol events.

3. The transport knows nothing about turns, threads or tools.
4. Each adapter declares a `capabilities` object (compaction mode, steering, subagents). The engine uses a feature when it is present, instead of reducing every agent to the lowest common denominator.
5. Every adapter passes the same conformance test suite: the fake, Codex and Claude run the same file.

Why it pays off

- **One file per agent.** A new provider is a new adapter; nothing else changes.
- **Free tests.** The fake agent is just another adapter, so M2–M7 run with no model and no API key.
- **Contained failures.** A crashed CLI becomes one `exit` event; the engine marks the turn interrupted and nothing above needs to know why.
- **Provider switching.** The thread never belonged to Codex or Claude, so it can move between them (M6).
- **Comparable results.** Every agent looks the same to the engine, so the same task can run on each and be measured — the base for evals and cross-provider review after v0.1.

Stack, ground rules and layout

Use TypeScript on Bun with `bun:sqlite`, the same stack as OpenBot, so you can compare files side by side. Use plain `async` / `await`; learn Effect later as a separate project.

Ground rules

1. Build the fake provider first. Every test runs against it: deterministic, free, and able to stall or crash on command. OpenBot does the same in `scripts/provider-fakes/`.
2. Write your version before you read theirs. Then read their file and note what they handled that you missed.
3. Every milestone ends with a test that proves its "Done when" line, and a small user-facing result.
4. SQLite is the source of truth. Provider session files are cache.
5. Never retry a turn whose outcome is unknown. Mark it `interrupted` and let a person decide.

Repository layout

```
kru/
  src/
    rpc/          JSON-RPC over stdio (M0)
    providers/    ProviderClient + adapters: fake, claude, codex (M1)
    store/        SQLite, events, projections, migrations (M2, M4)
    mailbox/      enqueue, delivery state machine (M2)
    drain/        lanes and scheduler (M3)
    turns/        notification handling, delta buffer (M4)
    tools/        tool router, send_message, history tools (M5, M6)
    threads/      provider sessions, handoff (M6)
    boot/         crash recovery (M7)
    api/          HTTP + SSE server (v1.0)
    cli/          kru command (every milestone)
  test/
    fakes/        scripted fake agents: echo, stall, crash, tool-caller
  examples/
  docs/
```

Core types

Five types carry the whole engine. Start with these and change them as the milestones teach you more.

```
// Who sent a delivery. Users, agents and routines share one queue.
type Sender =
  | { kind: "user" }
  | { kind: "agent"; agentId: string }
  | { kind: "routine"; routineId: string };

type DeliveryStatus = "queued" | "starting" | "running" | "completed" |
"failed" | "interrupted" | "cancelled";

// One unit of work for one agent. A row in SQLite, never only in memory.
interface Delivery {
  id: string;
  recipientAgentId: string;
  laneId: string | null;          // null = the agent's main chat
  sender: Sender;
  text: string;
  replyToMessageId: string | null;
  expectsReply: boolean;
```

```
  status: DeliveryStatus;
  idempotencyKey: string | null;  // UNIQUE: a retry returns the first
receipt
  createdAt: string;
}

// Kru's thread. Survives provider switches.
interface Thread { id: string; agentId: string; laneId: string | null }

// The provider's private session, bound to one thread. Disposable.
interface ProviderSession { threadId: string; provider: string;
externalSessionId: string; retiredAt: string | null }

// Every provider adapter implements this. One protocol for all of them.
interface ProviderClient {
  readonly provider: string;
  start(): Promise<void>;
  stop(): Promise<void>;
  request<T>(method: string, params: unknown, timeoutMs?: number):
Promise<T>;
  respond(id: string | number, result: unknown): void;
  respondError(id: string | number, error: { code: number; message: string
}): void;
  on(event: "notification", fn: (n: { method: string; params: any }) =>
void): void;
  on(event: "request", fn: (r: { id: string | number; method: string; params:
any }) => void): void;
  on(event: "exit", fn: (error: Error) => void): void;
}
```

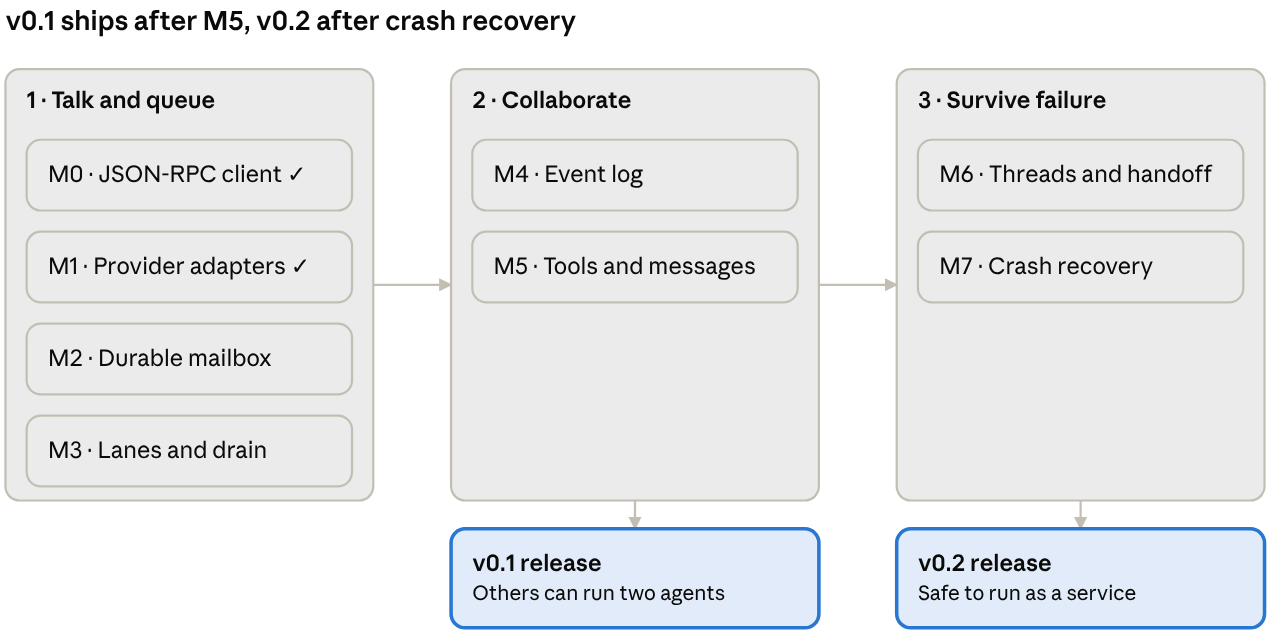
The protocol methods every adapter must speak:

Direction	Method	Meaning
Kru → provider	`thread/start`, `thread/resume`	Open or reopen a native session
Kru → provider	`turn/start`, `turn/interrupt`	Run one turn, or stop it
Provider → Kru	`turn/started`, `turn/completed`	Turn boundaries, with status

Direction	Method	Meaning
Provider → Kru	`item/started`, `item/agentMessage/delta`, `item/completed`	Streamed output and tool steps
Provider → Kru	`thread/tokenUsage/updated`	Context fill, for compaction
Provider → Kru (request)	`item/tool/call`	The model calls a Kru tool and waits for the answer

Milestones

Eight milestones, each isolating one idea. After M5 Kru is usable by other people (v0.1). After M7 it is crash-safe (v0.2). Each milestone lists what to build, which OpenBot files to read after you build it, the test that proves it, and what a user can do with it.



Kru roadmap · 8 milestones in 3 phases, 2 releases

Each phase depends on the one before it; inside a phase, build in order.

M0 — JSON-RPC over stdio (the transport) · Done

Status: done. Released as v0.0.1 on 2026-11-02, with the review fixes in v0.0.2 the same day. The spec below is what was built; the implementation record at the end of this milestone says how.

Purpose. Kru does not run a model. It starts agent CLIs as child processes and talks to them over stdin and stdout. M0 builds the bottom layer of the hexagon: a generic connection that sends and receives JSON-RPC messages reliably. It knows nothing about Codex, Claude, turns or threads, and it is proven against a fake agent. In M1 the Codex and ACP adapters use it directly; the Claude adapter uses the Agent SDK instead.

Public API

```
interface RpcConnectionOptions {
  command: string;
  args: string[];
  env?: Record<string, string>;
  requestTimeoutMs: number;
  maxLineBytes: number;        // one message = one line; longer = fatal
  maxQueuedWriteBytes: number; // stdin backpressure cap
  maxBadLines: number;         // consecutive unparsable lines before failing
}

class RpcConnection {
  constructor(options: RpcConnectionOptions);
  start(): void;
  stop(): Promise<void>;       // ends the whole process tree
  request<T>(method: string, params: unknown,
    options?: { timeoutMs?: number; signal?: AbortSignal; decode?: (value:
unknown) => T }): Promise<T>;
  notify(method: string, params?: unknown): Promise<void>;
  // The transport sends the answer: the resolved value, or an error if it
throws.
  onRequest(handler: (request: { id: string | number; method: string; params:
unknown; signal: AbortSignal }) => Promise<unknown>): void;
  on(event: "notification", listener: (n: { method: string; params: unknown
}) => void): void;
  on(event: "diagnostic", listener: (d: Diagnostic) => void): void; // stderr
| bad-line | late-response | write-blocked
  on(event: "exit", listener: (error: ProcessExitError) => void): void; //
fires exactly once
}
```

Design rules

1. **Pure transport.** No method names, turns or threads inside this module. Agent-specific behaviour, such as a liveness probe that needs a cheap method, belongs to the M1 adapter.
2. **Frame on bytes.** Split on the newline byte before decoding UTF-8, so a character split across two chunks is never corrupted.
3. **Every request gets exactly one answer.** The handler's value becomes the result; a thrown error becomes `-32603`; no handler becomes `-32601`. A deadlock from a missing listener is impossible.
4. **A bad line is a diagnostic, not a crash.** Messages before and after it in the same chunk are still handled. After `maxBadLines` consecutive bad lines the connection fails, so a truly broken CLI is still restarted.
5. **Backpressure.** When `stdin.write()` returns false, wait for `drain`. Queued bytes above `maxQueuedWriteBytes` reject the write with a clear error instead of growing memory.
6. **Late answers are visible.** Remember recent timed-out ids in a bounded set. An answer that arrives later becomes a `late-response` diagnostic with the method and the delay.
7. **Stale-process guard.** Events from an old child are ignored after a restart, and request ids never reset, so an old answer cannot resolve a new request.
8. **Exit reports the real reason.** On exit, wait briefly for stderr to end, then reject every pending request with a `ProcessExitError` that carries the last stderr line.
9. **Stop ends the whole tree.** Terminate the child and its children; force-kill after a grace period, so no MCP servers are left behind.
10. **One line has a size limit.** Past `maxLineBytes` the stream cannot recover, so the connection fails. OpenBot sets 256 MiB, because one Codex thread resume reached 21 MB.

Fake agent (`test/fakes/fake-agent.ts`) — a script that speaks the protocol and, on command: answers normally, answers out of order, stalls, crashes, sends notifications, sends a request back, writes split, merged or invalid lines, writes to stderr, and stops reading stdin. Every later milestone reuses it.

Tests — `bun test`, no network, no API key, about 2 seconds.

#	Test	Proves
1	`ping` returns `pong`	Basic round trip
2	100 concurrent requests answered in random order	Each answer reaches its own caller
3	Error response	Rejects with the code and message
4	Fake stalls	Rejects with a timeout error
5	Five notifications	Arrive in order
6	Fake sends a request; the handler answers	The fake receives the answer and echoes it back
7	One message split across two chunks; two messages in one chunk	Framing is correct
8	A bad line between two good lines in one chunk	Both good messages handled; one `bad-line` diagnostic; connection stays up
9	Fake exits during a request	Every pending request rejects; `exit` fires once, with the last stderr line
10	`stop()`, then `request()`	Child tree is gone; the new request rejects at once
11	Fake writes to stderr	Reported as a diagnostic, never parsed as protocol
12	Fake stops reading; 50 MB of writes	Memory stays bounded; writes wait or fail cleanly
13	Timeout, then the fake answers late	One `late-response` diagnostic; no crash, no double resolve
14	Fake sends a request with no handler set	It receives a `-32601` error

After every test: the pending map is empty, no child process is running, and there are no unhandled promise rejections.

Done when: all 14 tests and the hygiene checks pass, every time.

You can now:

```
$ kru rpc ping --fake
pong (1.4 ms)

$ kru rpc ping --fake --stall
error: request "ping" timed out after 2000 ms
```

Stretch: send `initialize` to a real `codex app-server --listen stdio://` and print the answer.

Read after: `src/backend/app-server-client.ts`, `src/backend/jsonl.ts`, `src/backend/protocol.ts`.

Out of scope: providers, SQLite, queues, retries.

Improvements over OpenBot

OpenBot's transport is production-hardened. Kru keeps its byte framing, line limit, stale-process guard, stderr-before-exit and process-tree kill, and changes six things:

Area	OpenBot	Kru M0
Separation	The Codex client mixes transport with Codex logic (turn interrupts, resume params, history paging); ACP and Pi have their own transports	One generic transport for every JSON-RPC agent; agent logic lives in adapters
Bad line	Any unparsable line ends the process and every agent on it; valid messages earlier in the chunk are dropped	Diagnostic and skip; fail only after `maxBadLines` in a row
Backpressure	`stdin.write()` result ignored; writes queue without limit	Wait for `drain`; cap queued bytes
Late answers	Ignored silently after a timeout	Reported as `late-response`, so the engine can reconcile a turn that did start
Unanswered requests	The tool router answers unknown methods, but the transport does not guarantee an answer	The transport always answers exactly once

Area	OpenBot	Kru M0
Types	`params: unknown`, checked deep in the engine	Decoded at the boundary through `decode`

One trade-off: OpenBot's fail-fast on a bad line is a deliberate safety choice. Skipping is safe here only because each NDJSON line is independent, which is why the `maxBadLines` limit stays.

Implementation record

The M0 transport meets every design rule above and all six improvements over OpenBot. Code: aminhaiqal/kru; exports in `src/rpc/index.ts`, tests in `test/rpc.test.ts`.

Shipped in v0.0.1

- Generic JSON-RPC connection: byte framing, strict UTF-8, CRLF accepted, incoming and outgoing line limits, concurrent requests, notifications, inbound request handlers.
- Timeouts, cancellation and result decoding; bounded stdin writes with `drain`; a `late-response` diagnostic from a bounded set of the last 1,024 timed-out ids.
- Process-tree cleanup: each child in its own process group on POSIX, SIGTERM then SIGKILL; `taskkill /T /F` on Windows.
- The deterministic fake agent, `kru rpc ping --fake` with stall mode, and `kru --version`.
- 32 tests and GitHub Actions CI.

Beyond the plan

- An oversized outgoing frame rejects only its own write, not the connection.
- Cancelling or timing out a queued request removes its unsent frame.
- Handler replies from a previous process are suppressed after a restart.
- Each spawned process owns its own listeners, maps, buffers and handler controllers.
- Read-only `pid`, `pendingRequestCount` and `queuedWriteBytes` for observability.

Review fixes (v0.0.2)

Found in review	Fix
Codex omits the `jsonrpc` field; a strict check would turn every Codex message into a bad line	Accept a missing field, reject a wrong version, and set outgoing frames with the `jsonrpcField` option (default `true`); a Codex connection factory passes `false`
A 100 ms SIGTERM grace can corrupt a real agent's session files	Configurable shutdown grace, default 3 s; configurable stderr wait, default 500 ms; tests keep shorter values
Children in their own process group no longer receive Ctrl+C	Every registered connection stops on SIGINT or SIGTERM; handlers stay until cleanup ends; the CLI returns conventional signal exit codes
A hung shutdown makes the user wait out the grace period	A second signal sends SIGKILL to every tracked process group at once, skips the remaining waits, reaps the children and exits
After `kill -9` of Kru, children run on as orphans	Each child's PID and process group are saved with its start identity, for M7 recovery, and removed after cleanup
A crash during a write could leave a half-written ownership record	Records are published with an fsynced temporary file and an atomic rename; the reader ignores unpublished, truncated and invalid records
macOS can refuse a process-group signal during reaping	The transient permission error is handled

Verification

- Strict TypeScript checking passes.
- All 32 v0.0.1 tests pass in about 1.2 s with Bun 1.4.4 on macOS; ten repeated runs pass all 320 cases with the per-test hygiene checks.
- Ten repeated suite runs in Linux CI, with manual workflow dispatch.
- An exact-byte `initialize` smoke test passes against real Codex 0.165.0, locally and in CI.
- v0.0.2 tests cover envelope compatibility, graceful stop, the second-signal escalation, process ownership, the ownership reader and the signal lifecycle.

Open items

- [x] ~~Tag v0.0.2 before M1 starts~~
- [x] ~~A second Ctrl+C force-kills every tracked process group and exits~~
- [x] ~~Write the ownership file atomically; ignore truncated and invalid records~~
- [x] ~~Real Codex `initialize` smoke test with `jsonrpcField: false`~~
- [ ] Confirm the `late-response` diagnostic names the method as well as `delayMs`
- [ ] Validate process-tree cleanup on Windows

M1 — One protocol, many providers (the port and adapters) · Done

Status: done on 2026-11-03. Fake, Codex and Claude adapters run behind one typed port and pass the same conformance suite. Five follow-ups and the v0.0.3 tag are listed at the end of the implementation record, below the improvements table.

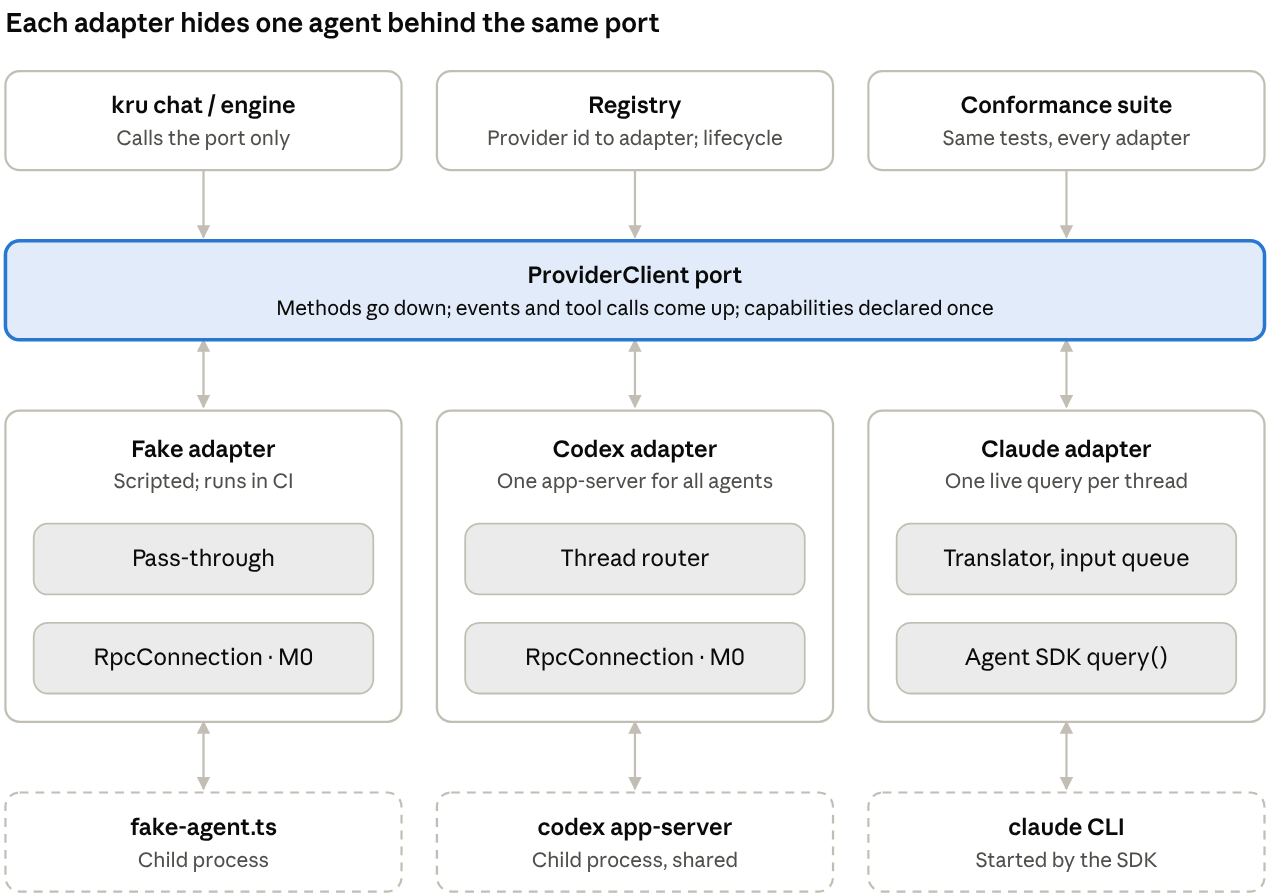
Purpose. M1 builds the `ProviderClient` port and the adapters behind it. After M1 the engine runs a turn on the fake, Codex or Claude without knowing which. M1 is also where token use becomes visible: every adapter must report what each turn cost, because the main reason people leave OpenBot is how many tokens it uses.

Build order

1. Typed protocol: one TypeScript map of methods (params and result) and one of events. The method names stay those in Core types.
2. A `capabilities` object that every adapter declares.
3. The `ProviderClient` port and a registry that picks the adapter.
4. The fake adapter on the M0 fake agent, with scripted token counts.
5. The conformance suite, written against the fake first.
6. The Codex adapter on `RpcConnection` through the existing Codex factory (`jsonrpcField: false`). One app-server process serves every Codex agent as separate threads.
7. The Claude adapter around the Agent SDK. It is the thick one, so it comes last.
8. The `kru chat` demo, which prints the usage of each turn.

Architecture

M1 has three parts: the port (the contract), the adapters (translation), and the registry with the conformance suite (wiring and proof).



M1 architecture · port, registry, conformance suite, 3 adapters, 3 processes

Commands go down; events and tool calls come up. Fake and Codex reuse the M0 transport; Claude goes through the Agent SDK.

The port carries four kinds of traffic

- **Down, methods:** `thread/start`, `thread/resume`, `turn/start`, `turn/interrupt`.
- **Up, events:** `turn/started`, message deltas, `item/completed`, usage, `turn/completed`.
- **Up, one request:** a tool call, which the engine's handler must answer.
- **Declared once:** `capabilities`.

Inside an adapter

Every adapter has the same four jobs:

1. Translate methods down into the agent's native calls.
2. Translate native output up into protocol events.

3. Bridge tool calls: the native tool call goes to `onToolCall(call, signal)`, and the result goes back in native form.
4. Normalise usage and errors.

An adapter keeps two pieces of state only: Kru's thread id mapped to the native session, and the active turn of each thread. A turn moves from `starting` to `running` to `completed`; a process exit from any state ends it `interrupted`.

How the three adapters differ

	Fake	Codex	Claude
Transport	M0 `RpcConnection`	M0 `RpcConnection`, `jsonrpcField: false`	Agent SDK in process; it starts the `claude` CLI
Processes	One per test	One app-server for every Codex agent	One live `query()` per active thread, closed when idle
Start / resume	Passed through	`thread/start` with instructions and tools / `thread/resume`	`query({ sessionId })` / `query({ resume })`
One turn	`turn/start`	`turn/start`	A message pushed into the thread's input queue; the turn ends at the SDK result message
Text streaming	Passed through	`item/agentMessage/delta`	SDK stream events (`includePartialMessages: true`)
Kru tools	Scripted	Declared at thread start; Codex sends `item/tool/call`	An in-process MCP server (`createSdkMcpServer`); the model sees `mcp__kru__send_message`
Usage source	Scripted	`thread/tokenUsage/updated`, last-turn value	SDK result message: input, cache read, cache write, output
Size	Trivial	Thin	Thick

Claude: streaming input

The Claude adapter does not start a new `query()` for each turn. It starts one per thread whose prompt is an async queue, and each `turn/start` pushes a message into it. The session stays alive between turns: no process restart, and the prompt cache stays warm. OpenBot does the same with `AsyncMessageQueue` in `src/backend/claude-client.ts`.

The tool-call path

1. The model calls a Kru tool; the adapter receives the native call and calls `onToolCall(call, signal)`.
2. The engine's handler does the work — in M5, it enqueues a delivery — and returns a result.
3. The adapter sends the result back in native form, and the model continues.

As in M0, every tool call gets exactly one answer, and `signal` aborts when the turn is interrupted or the process exits.

Registry and lifecycle

- `registry.create(provider, options)` returns a `ProviderClient`. Nothing else in Kru constructs an adapter.
- Adapters start on first use. The registry hands out one shared Codex instance.
- On a signal, the registry stops every adapter through the M0 signal handling.

The conformance suite (below) runs against `registry.create("fake" | "codex" | "claude")`. It is the executable definition of the port: an adapter that passes it works with the engine.

Folder layout

```
src/providers/
  protocol.ts          typed method and event maps, capabilities (depends on
nothing)
  port.ts              ProviderClient interface
  registry.ts          provider id → adapter factory, lifecycle
  fake/adapter.ts
  codex/adapter.ts     uses src/rpc (M0)
  claude/adapter.ts    uses @anthropic-ai/claude-agent-sdk
  claude/input-queue.ts
test/providers/conformance.test.ts
```

Dependencies point inward: adapters import `protocol.ts` and their own transport; the engine imports `port.ts` and `registry.ts` only.

Port API

```
interface ProtocolMethods {
  "thread/start":   { params: { cwd: string; instructions: string; tools:
ToolDefinition[]; model: string;
                      promptProfile: "minimal" | "coding" | "full";
                      permissions: { sandbox: "read-only" | "workspace-
```

```
write"; approvalPolicy: "on-request" | "never" } }; result: { threadId:
string } };
  "thread/resume":  { params: { threadId: string }; result: { threadId:
string } };
  "turn/start":     { params: { threadId: string; input: string; model:
string }; result: { turnId: string } };
  "turn/interrupt": { params: { threadId: string; turnId: string }; result:
Record<string, never> };
}

interface ProtocolEvents {
  "turn/started":              { threadId: string; turnId: string };
  "item/agentMessage/delta":   { threadId: string; turnId: string; itemId:
string; delta: string };
  "item/completed":            { threadId: string; turnId: string; item:
ThreadItem };
  "thread/tokenUsage/updated": { threadId: string; turnId: string;
inputTokens: number; cachedInputTokens: number;
                                outputTokens: number; contextWindow: number
| null; estimated: boolean };
  "turn/completed":            { threadId: string; turnId: string; status:
"completed" | "failed" | "interrupted"; error?: string };
}

interface ProviderCapabilities {
  compaction: "events" | "request" | "native" | "handoff" | "none";
  steer: boolean;               // can add input to a running turn
  multiThreadProcess: boolean;  // one process serves many threads
  usage: "exact" | "estimated";
  promptCache: boolean;
}

interface ProviderClient {
  readonly provider: string;
  readonly capabilities: ProviderCapabilities;
  start(): Promise<void>;
  stop(): Promise<void>;
  request<M extends keyof ProtocolMethods>(method: M, params:
ProtocolMethods[M]["params"]): Promise<ProtocolMethods[M]["result"]>;
  on<E extends keyof ProtocolEvents>(event: E, listener: (params:
ProtocolEvents[E]) => void): void;
```

```
  onToolCall(handler: (call: ToolCall, signal: AbortSignal) =>
Promise<ToolResult>): void;
  onExit(listener: (error: Error) => void): void;
}
```

Design rules

1. The engine imports the port and the registry only, never an adapter.
2. An adapter never writes to SQLite or the screen. It only emits protocol events.
3. A turn that ends normally ends with exactly one `turn/completed`. If the process dies, the adapter emits `exit` and does not invent a completion: it does not know how the turn ended. The waiting turn rejects with an unknown outcome, and the engine decides what it was (M7).
4. Every turn reports usage. When the provider gives counts, `estimated` is false; when it does not, the adapter estimates (about 4 bytes a token) and sets `estimated: true`. A turn with no usage event fails conformance.
5. A stable prefix. Instructions and tool definitions are fixed when the thread starts and never injected per turn, so the provider's prompt cache stays valid. Changing them starts a new session, and the reason is logged.
6. A small surface. Kru's own instructions stay under 8 KB (about 2,000 tokens) and an agent sees at most 6 tools in M1. A test fails when either grows, as OpenBot's `check:agent-context` does, but with a lower limit.
7. Shared processes. When `multiThreadProcess` is true (Codex), one process serves all agents of that provider. Claude threads with no turn for 10 minutes are closed and resumed on demand.
8. Capabilities, not the lowest common denominator. The engine uses a feature when an adapter declares it.
9. A prompt profile per agent. `minimal` is a short custom prefix with no built-in tools. `coding` is a short coding prefix with read tools for read-only agents or coding tools for writable ones. `full` keeps the agent CLI's own system prompt, project settings and standard tools, and is the baseline for measuring the saving (a follow-up). For Claude, `minimal` and `coding` use a custom `systemPrompt`, `settingSources: []` and an explicit tool list.

Conformance suite — one test file; every adapter runs it.

#	Test	Proves
1	Start a thread, run a turn	Deltas arrive, then one `turn/completed`
2	Interrupt during a turn	Status `interrupted`; still exactly one `turn/completed`
3	The model calls an `echo` tool	The handler's result reaches the model
4	The process exits during a turn	`turn/completed` with `interrupted`, then `exit`
5	Restart the adapter and resume the thread	The next turn works on the same thread
6	Usage	One or more usage events per turn; counts are not negative; `estimated` matches `capabilities.usage`
7	Prefix stability	Two turns on one thread send identical instructions and tool definitions
8	Context budget	Instructions under 8 KB; 6 tools or fewer

The fake runs the suite in CI on every push. Codex and Claude run it in a nightly canary on the cheapest model, because it needs keys and costs tokens.

Watch for: providers that stream text before they announce the item — create the message on the first delta. Codex reports usage as both a last-turn and a running total; take the last-turn value, never add the totals.

Done when: `kru chat` works the same with `--provider fake`, `codex` and `claude`; the conformance suite passes for all three; and every turn prints its usage.

You can now (example output):

```
$ kru chat --provider claude
> summarise this repository
...
[turn 1 · input 12,410 tokens (10,880 cached) · output 642]
```

Read after: `src/backend/agent-client.ts`, `src/backend/claude-client.ts` (the method switch near line 494 and the functions that emit `item/*`), `src/backend/acp-client.ts`, `src/backend/provider-drivers.ts`, `src/backend/agent/usage-`

`collection.ts`, `scripts/provider-fakes/agents/conformance.ts`, `tools/agent-context/budget.json`.

Improvements over OpenBot

Area	OpenBot	Kru M1
Types	String methods; `params: unknown` read with `getString`	Typed method and event maps; the compiler checks each adapter
Capabilities	Optional fields spread over the client	One declared object
Usage	Collected from provider-specific notifications in the turn lifecycle	A required event; every adapter emits it, exact or estimated
Added context	About 14 KB of instructions and 14 KB of visible tools — roughly 7,000 tokens on every model step, before memories and teammates	Under 8 KB of instructions and at most 6 tools, enforced by a test
Per-turn text	ACP and Muse insert instructions on each normal turn	Fixed at thread start, never per turn
Claude system prompt	Always the full `claude_code` preset with OpenBot's instructions appended	`minimal` or `coding` per agent, with `full` as the baseline; the saving is measured
Conformance	A fake agent per scenario	One suite that every adapter must pass

Implementation record

Code: aminhaiqal/kru; port in `src/providers/protocol.ts`, adapters in `src/providers/{fake,codex,claude}/`, turn consumer in `src/chat/turn.ts`.

Delivered

- **Typed port.** Four methods with checked params and results. Typed notifications for turn boundaries, items, deltas, usage, tool calls and `approval/request`. Every tool or approval request gets exactly one reply, with a deadline and an abort signal. Request ids are never reused, even after a restart.

- **Fake adapter.** `echo`, `stall`, `crash`, `failure` and `tool` modes; deterministic usage values for tests.
- **Codex adapter.** One shared `CodexRuntime` (one app-server) for every agent, with thread ownership that routes events, tool calls and approvals to the right agent. Stopping one agent leaves the others running; the last one stops the process. Uses the experimental dynamic-tool API and `jsonrpcField: false`.
- **Claude adapter.** A custom `systemPrompt`, `settingSources: []` and an explicit built-in tool list. An in-process MCP server for Kru's tools. One persistent streaming input queue per thread. Invented turn ids. Per-turn `usage` kept apart from the session's cumulative `modelUsage`. Resume through the SDK's session metadata.
- **Agent configuration.** `cwd`, `model`, `instructions`, `tools`, `promptProfile` and `permissions` (default `read-only` and `on-request`), also as CLI flags.
- **Prefix control.** Stable profile text, canonical tool order and a snapshot at thread start. `Thread.prompt` reports the profile, instruction bytes, tool count and a SHA-256 prefix hash. Instructions of 8,000 bytes or more and a seventh tool are refused at start and at resume.
- **Approvals.** A missing handler declines; nothing hangs and nothing is granted by default. The CLI declines every request.
- **Fault isolation.** A malformed known notification fails only its turn; three in a row end the process.
- **Chat.** An idle timeout (default 120,000 ms) that resets on activity from the same thread. Usage on stderr after each turn, or `usage: unavailable`.
- **Shutdown.** M0's signal handling for all providers. Claude subprocesses also run in process groups with ownership records.

Changes from the plan

Plan	Built	Why
Rule 3: the adapter emits `turn/completed` with `interrupted` when the process dies	`exit` only; the turn's outcome stays unknown	The adapter cannot know how the turn ended. Rule 3 now says so.
`promptProfile`: `full` or `lean`	`minimal` and `coding`, both short custom prefixes	`full` is a follow-up, needed as the measuring baseline

Plan	Built	Why
At most 6 Kru tools	For Claude, at most 6 built-in and Kru tools together	Leaves no room for M5's tools; a follow-up separates the budgets
A wall-clock turn timeout	An idle timeout	A turn that keeps making progress can run as long as it needs

Verification

- The same conformance cases, including #7 (prefix stability) and #8 (budget), run against all three adapters. Codex runs against a scripted stdio peer; Claude against SDK events and a real in-memory MCP transport.
- Ten repeated offline runs: 1,020 passes, no failures. Strict TypeScript and a frozen install pass.
- Opt-in live tests pass on real Codex and Claude: streaming, usage, resume, tool calls and interruption through the common port.
- `codex-canary.yml` runs nightly at 03:17 UTC against the latest Codex. It checks the `initialize` bytes, dynamic-tool registration and the generated protocol schemas, with no model call. Validated locally against Codex 0.166.0; pinned CI stays on 0.165.0.

Follow-ups

- [x] ~~Normalise usage across providers. Claude reports cache reads and writes apart from `input_tokens`; Codex counts cached tokens inside its input. Report `inputTotal`, `cachedRead`, `cacheWrite` and `output`; print the cached part in the CLI; add a conformance check that `inputTotal` is at least `cachedRead + cacheWrite`.~~
- [x] ~~Separate the tool budgets: at most 6 Kru-declared tools; built-in tools chosen by the profile, reported but not capped.~~
- [x] ~~Add the `full` profile (the `claude_code` preset and project settings), then compare `full` and `coding` on one task for tokens and result.~~
- [x] ~~Permission parity: `workspace-write` on Claude allows file tools inside `cwd` and its roots and asks only for shell commands or outside paths. Add a conformance case: the same settings allow and refuse the same actions on every adapter.~~
- [x] ~~Confirm that output or progress from a long shell command resets the idle timer on both adapters.~~
- [x] ~~Merge the canary to the default branch so its schedule runs.~~

- [x] ~~Tag v0.0.3.~~

M2 — The durable mailbox

Purpose. M2 makes Kru durable. The mailbox is a SQLite inbox through which all work enters — from a person, another agent or a routine. M2 only stores work and moves it between states; it never runs a turn. Running is M3's job. Keeping the two apart is what lets M7 recover after a crash.

Architecture

- `src/store/db.ts` opens `bun:sqlite` with WAL mode, a busy timeout and foreign keys on, and runs migrations from version 1.
- `src/mailbox/` holds the mailbox API, the delivery state machine and the queue queries.
- `src/agents/` is a minimal agent table, so `enqueue` can check that a recipient exists.
- The CLI adds `kru agent create`, `kru send`, `kru queue` and `kru cancel`.

The mailbox is a port, as the provider is in M1. M4 replaces its internals with the event log without changing a caller.

Data model: messages apart from deliveries

```
CREATE TABLE agents (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL UNIQUE,
  provider    TEXT NOT NULL,
  config_json TEXT NOT NULL CHECK (json_valid(config_json)),
  created_at  TEXT NOT NULL
);

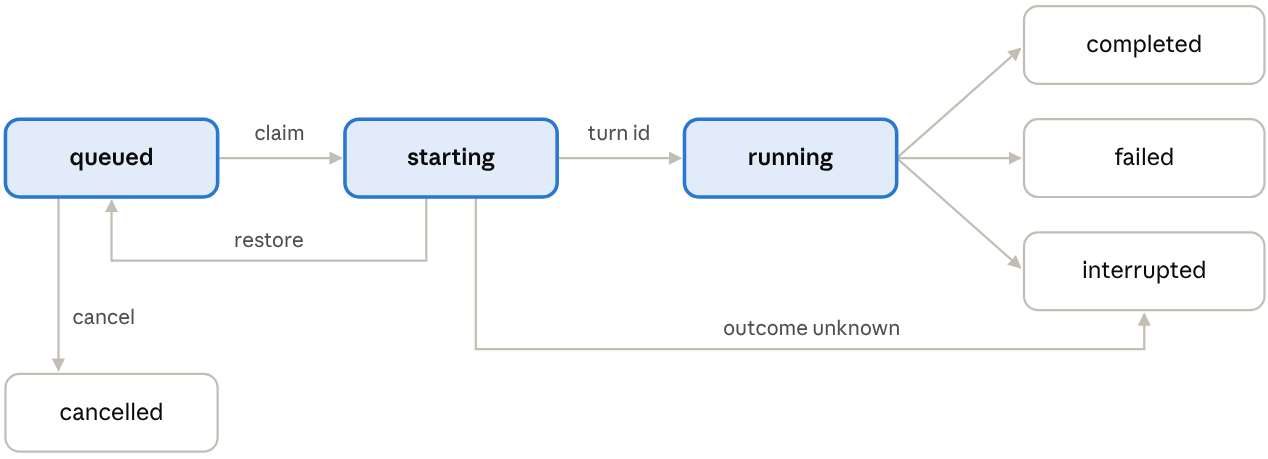
CREATE TABLE messages (
  id              TEXT PRIMARY KEY,
  sender_kind     TEXT NOT NULL CHECK (sender_kind IN
('user','agent','routine')),
  sender_agent_id TEXT REFERENCES agents(id),
  text            TEXT NOT NULL,
  reply_to        TEXT,
  idempotency_key TEXT UNIQUE,          -- NULL allowed; duplicates refused
  request_hash    TEXT,                 -- same key with other content =
client error
  created_at      TEXT NOT NULL
);
```

```
CREATE TABLE deliveries (
  seq          INTEGER PRIMARY KEY,     -- queue order; never order by time
  id           TEXT NOT NULL UNIQUE,
  message_id   TEXT NOT NULL REFERENCES messages(id),
  recipient_id TEXT NOT NULL REFERENCES agents(id),
  lane_id      TEXT,                    -- NULL = the agent's main chat
  status       TEXT NOT NULL,
  turn_id      TEXT,
  error        TEXT,
  retry_of     TEXT REFERENCES deliveries(id),
  chain_id     TEXT NOT NULL,           -- M5 hop limit; in the schema from
version 1
  hop          INTEGER NOT NULL DEFAULT 0,
  updated_at   TEXT NOT NULL
);
CREATE INDEX deliveries_queue ON deliveries (recipient_id, lane_id, status,
seq);
```

One message can go to several agents: `send_message` to B and C is one message with two deliveries, each with its own status. OpenBot splits them the same way (`projection_mailbox_messages` and `projection_deliveries`). The auto-increment `seq` gives a strict order; two messages can share a millisecond, and clocks can jump.

The delivery state machine

A delivery only moves forward; final states never reopen



Tinted: open states. Outlined: final states; a retry creates a new delivery.

Delivery state machine · 3 open states, 4 final states

A drain claims a queued delivery into `starting`; a confirmed turn id moves it to `running`. If `turn/start` fails before a turn begins, the delivery is restored to `queued` at its original place, or ends `failed` when the provider refused it. If a turn may have begun, it ends `interrupted` and is never retried automatically.

Mailbox API

```
type DeliveryStatus = "queued" | "starting" | "running" | "completed" |
"failed" | "interrupted" | "cancelled";

interface EnqueueInput {
  sender: Sender;
  recipientIds: string[];
  text: string;
  laneId?: string | null;
  replyTo?: string | null;
  idempotencyKey?: string;
  chain?: { id: string; hop: number };   // set by M5; otherwise a new chain
}

interface EnqueueReceipt { messageId: string; deliveryIds: string[];
duplicate: boolean }

interface Mailbox {
  enqueue(input: EnqueueInput): EnqueueReceipt;
  claimNext(agentId: string, laneId: string | null): Delivery | null; //
queued → starting
  markRunning(deliveryId: string, turnId: string): void;              //
starting → running
  finish(deliveryId: string, status: "completed" | "failed" | "interrupted",
error?: string): void;
  restore(deliveryId: string): void;                                  //
starting → queued
  cancel(deliveryId: string): void;                                   //
queued → cancelled
  retry(deliveryId: string): EnqueueReceipt;                          //
final → a new delivery
  get(deliveryId: string): Delivery | null;
  listQueue(agentId: string, laneId?: string | null): Delivery[];
  onChange(listener: (change: { deliveryId: string; status: DeliveryStatus })
=> void): () => void;
}
```

Design rules

1. Store and transition only. The mailbox never runs a turn.
2. Every transition is a compare-and-set update, checked against one table of allowed transitions: `UPDATE deliveries SET status = ? WHERE id = ? AND status = ?`. A change count other than 1 means another writer won. This is what lets M3 run several drains without claiming the same work twice.
3. A final state never reopens. A retry creates a new delivery with `retry_of`, so the history shows that the first attempt failed.
4. Idempotency in the same transaction as the insert: a `UNIQUE` key plus a hash of the request. A duplicate returns the first receipt with `duplicate: true`; the same key with other content is refused.
5. Notify after commit. `onChange` fires only after the transaction commits, so M3 never schedules work that was rolled back.
6. Queue order is `seq`, never a timestamp. `restore` keeps the delivery's `seq`, so it does not move to the back of the queue.
7. One writer at a time. WAL mode, `BEGIN IMMEDIATE` for every write and a busy timeout. The API is synchronous, as `bun:sqlite` is, so no mailbox operation can be interleaved halfway through.
8. Migrations from version 1. Each schema change and its `schema_migrations` row commit in one transaction. Every shipped version keeps a fixture, and a parity test checks that a new database and a migrated one have the same schema.
9. Fields for later milestones now: `chain_id` and `hop` for M5; `sender_kind` is the turn cause for M4.

Tests

#	Test	Proves
1	The same key sent twice	One message; the second call returns the first receipt with `duplicate: true`
2	The same key with other text	Refused; nothing written
3	One message to two agents	Two deliveries with independent statuses
4	Restart	Queued rows and their order survive
5	`completed → running`	Refused; the row does not change

#	Test	Proves
6	Two connections call `claimNext` at the same time	They never claim the same delivery
7	`claimNext` on one lane	Takes the lowest `seq` of that lane; other lanes do not change
8	`restore`	Back to `queued` with the original `seq`
9	`cancel`	Works on `queued`; refused on `running`
10	`retry` of a final delivery	A new delivery with `retry_of`; the old one does not change
11	A transaction that rolls back	No `onChange` event
12	Migrations	An empty database and a version-1 fixture end with the same schema
13	An unknown recipient	Refused; nothing written

Done when: all 13 tests pass in ten repeated runs, as in M0 and M1.

Watch for: `UPDATE … RETURNING` needs SQLite 3.35 or later — check the version that Bun bundles. A CLI and the M3 daemon can write at the same time; without a busy timeout, one of them gets `SQLITE_BUSY`.

Optional bridge to M3: `kru deliver --once <agent>` claims one delivery, runs it with the M1 `runChatTurn` and records the outcome. It is the smallest path from message to turn to final state; M3 automates it.

You can now (example output):

```
$ kru agent create reviewer --provider claude
$ kru send reviewer "review the latest commit"
queued  message 7f3c…  delivery d-1
$ kru queue reviewer
d-1  queued  user  "review the latest commit"
```

Read after: `src/backend/mailbox-store.ts` (`enqueue`, `markStarting`, `restoreQueued`, `markTerminal`), the `projection_mailbox_messages` and `projection_deliveries` tables in `src/backend/openbot-database-schema.ts`, "Database migrations" in `src/backend/AGENTS.md`, and `src/backend/openbot-database-schema-parity.test.ts`.

Improvements over OpenBot

Area	OpenBot	Kru M2
Scope	`mailbox-store.ts` is about 3,300 lines: deliveries, attachments, reactions, edits and a file-deletion outbox	Deliveries only; attachments get their own module later
Idempotency	Keys in the store's in-memory state map, pruned by age	A `UNIQUE` column written in the same transaction, and a check that the content matches the key
Transitions	Status checks spread through the store	One allowed-transition table and compare-and-set updates
Retry	The same delivery is requeued	A new delivery that links to the old one
Chain and hop fields	Not part of the design	In the schema from version 1

M3 — Lanes and the drain scheduler

This is the most important milestone. The scheduler decides what runs, when, and in which order.

- **Build:** `scheduleDrain(agentId)` deduplicated by a "scheduled" set and a "draining" set. One turn at a time per lane (an agent's main chat, or one side thread). After a turn ends, schedule the lane again. Batch consecutive user messages into one prompt.
- **Build — token controls:**
  - **Budget hold.** A lane does not start a turn when its agent's daily token budget is spent. The lane reports the hold reason `budget`; queued work stays queued. In M3 the counter is kept in memory from M1 usage events; M4 makes it durable.
  - **Concurrency cap.** At most N turns run at once across all agents (default 2), so a burst of work cannot start every CLI together.
  - **No automatic turns.** A turn that ends with open plan steps does not start another turn. Every turn has a delivery behind it.
  - **Batching is the main saving.** Each turn resends the agent's whole context, so five messages in one turn cost about one fifth of five turns.

- **Read after:** `src/backend/agent/drain-scheduler.ts` — `#scheduleLane`, `#drainLane`, `startDelivery`; `src/backend/agent/turn-slots.ts`; `src/backend/agent/usage-limit-gate.ts`.
- **Done when:**
  - five fast messages give one running turn, then one batched turn for the other four;
  - two agents run in parallel, and no turn starts twice;
  - a spent budget holds the lane with reason `budget`, and raising the budget resumes it;
  - with the cap at 2, a third agent waits and starts when a slot frees;
  - no turn starts without a delivery.
- **Watch for:** the lost wakeup. A message that arrives while a drain finishes is never run. OpenBot's `Effect.ensuring` block checks `nextQueued` again after it clears the draining flag. The same applies to holds: when a budget resets or a slot frees, schedule the held lanes again.
- **Token note:** OpenBot runs up to 3 automatic plan continuations (`PlanContinuation({ limit: 3 })`). Kru runs none unless a person or a routine asks.
- **You can now:** `kru daemon` runs agents in the background while you queue work, and `kru agent set <agent> --daily-tokens 200000` caps what an agent can spend.

M4 — Event log and projections

- **Build:** `dispatch(commandId, events, project)` in one `BEGIN IMMEDIATE` transaction, with a command-receipts table. Stream deltas to listeners from memory; persist a message at `item/completed` or on a buffered flush.
- **Build — cost ledger:** a `usage_records` table with one row per turn: agent, thread, turn, provider, model, input, cached input and output tokens, `estimated`, and the **cause** of the turn (`user`, `agent`, `routine` or `system`) with the delivery and chain that caused it. Write it in the same transaction as the turn's completion. The M3 budget hold reads its daily totals from this table, so budgets survive a restart.
- **Read after:** `src/backend/database/database-core.ts` (`dispatch`), `src/backend/agent/delta-buffer.ts`, `handleNotification` in `src/backend/agent/turn-lifecycle.ts`, `src/backend/agent/usage-collection.ts`, `src/backend/database/agent-usage.ts`.
- **Done when:**

    - replaying the events table rebuilds identical projections;
    - a command run twice returns its stored result;
    - every turn has exactly one usage row, with a cause;
    - `kru cost` totals equal the sum of the usage events in the event log.
- **Watch for:** a nested `BEGIN` — open a transaction only when none is open. Several usage events can arrive for one turn: store the turn's final value, never the sum of the events.
- **You can now:** `kru log <agent>` shows full history after a restart, and `kru cost` shows where the tokens went (example output):

```
$ kru cost --by cause --since 7d
cause     turns   input     cached    output
user         42   510,200   431,900   38,120
agent        17   205,400   160,010   11,300
routine       9    88,900    80,200    2,100
```

M5 — Tools and agent-to-agent messages

- **Build:** route the provider's `item/tool/call` requests. Implement `send_message` as `mailbox.enqueue({ sender: { kind: "agent" } })` with the key `threadId + turnId + callId`. Add `list_agents`. These two are the only Kru tools in M5; no tool discovery.
- **Build — token controls:**
    - **Hop limit.** Every delivery carries a chain id and a hop count. A message an agent sends joins the chain of the turn that sent it, one hop higher. Past `maxHops` (default 4), `send_message` returns a clear tool error, so the model stops instead of passing work around.
    - **Replies are opt-in.** `expectsReply` defaults to false. An answer to a request never asks for an answer back.
    - **Visible cost.** Each chain's turns carry the chain id into the M4 ledger, so `kru cost --by chain` shows what one exchange cost.
- **Read after:** `handle` and `#sendMessage` in `src/backend/agent/openbot-tool-router.ts`, `src/backend/agent/tool-call-idempotency.ts`, `src/backend/agent/delivery-content.ts`.
- **Done when:**

  - agent A asks agent B, A's turn ends without waiting, and B's answer reaches A as a new delivery;
  - an acknowledgement loop caused on purpose (A thanks B, B thanks A) stops, because replies are off by default;
  - a chain that reaches `maxHops` ends with a tool error and starts no further turns;
  - `kru cost --by chain` shows the turns and tokens of one A↔B exchange.
- **Token note:** in OpenBot every message and every reply is a full turn for the recipient, with its whole context. One question between two agents costs at least two turns. The hop limit and opt-in replies bound how many turns one request can cause.
- **You can now:** create a reviewer agent and a coder agent that hand work to each other, within a known cost. Release **v0.1**.

M6 — Threads apart from provider sessions

- **Build:** your `threads` table and a `provider_sessions` binding table. On a provider switch or a missing session, start a new native session and prepend a handoff packet under a character budget. Add a `history_search` tool for what the handoff cut.
- **Read after:** `ensureThread` in `src/backend/agent/thread-lifecycle.ts`, `src/backend/agent/handoff-history.ts`, "History retrieval" in `docs/architecture/providers.md`.
- **Done when:** after ten turns on Claude you switch the agent to Codex, and it still answers questions about the first turn.
- **You can now:** `kru agent set <agent> --provider codex` with no lost context.

M7 — Crash recovery

- **Build:** at boot, before providers start, mark every `starting` or `running` delivery and every streaming message `interrupted`. Never retry them automatically. Let queued work continue.
- **Read after:** `src/backend/agent/boot-recovery.ts`.
- **Done when:** you `kill -9` the daemon during a turn of the stalling fake, start it again, and the state is consistent and the queue continues.
- **You can now:** run Kru as a service and trust it. Release **v0.2**.

Road to v1.0

After M7 the engine is complete. v1.0 adds what makes Kru useful to people who never read its code: an API, scheduled work, and limits that keep it safe on a shared machine.

Feature	What it adds	Read in OpenBot
HTTP + SSE API	Send, list, stream and cancel over HTTP, so any tool can drive Kru	`src/main/team-api-server.ts` (shape only)
Routines	A timer or webhook that enqueues a delivery; no new execution path	`src/backend/agent/routine-scheduler.ts`, `docs/architecture/events.md`
Context compaction	Ask the provider to compact near 80% of the window; rotate the session when it cannot	`src/backend/agent/context-compaction.ts`
Global turn slots	A cap on concurrent turns, people's messages first	`src/backend/agent/turn-slots.ts`
Deferred tool discovery	`tool_search` and `tool_call` keep the always-loaded tool list small	`src/backend/agent/tool-catalog.ts`
Agent memory	`remember`, `search_memories`, `forget_memory` tools on a SQLite table	`src/backend/agent/agent-memories.ts`
Workspace sandbox	Each agent writes only to its own folder plus allowed roots	`src/backend/agent/workspace-sandbox.ts`
Usage limits	Hold a lane when the provider reports a limit; resume when it resets	`src/backend/agent/usage-limit-gate.ts`

Build the API first. It turns Kru from a CLI into a platform, and every later feature can be tested through it.

Release checklist

People adopt a tool they can install and see working in five minutes. Do these before you announce v0.1.

- [ ] Check that the name "kru" is free on npm and GitHub; short names are often taken, so a scoped package such as @<your-name>/kru is the fallback
- [ ] Add an MIT `LICENSE` and a note that the project is inspired by OpenBot and shares no code with it
- [ ] One-line install: `bunx kru` or `npm i -g kru`
- [ ] README with a 30-second demo GIF: two agents hand work to each other
- [ ] A quick start that works with the fake provider, so a reader needs no API key to try it
- [ ] `examples/`: coder + reviewer pair, a Slack bot over the HTTP API, a nightly routine
- [ ] `docs/architecture.md` with the lifecycle diagram from this doc
- [ ] CI that runs every test against the fake providers
- [ ] A security note: agents run with the provider's own permissions; Kru adds no sandbox until v1.0
- [ ] Secrets are redacted from logs and stored error messages
- [ ] A `CHANGELOG.md` and semantic version tags
- [ ] A write-up of what you learned at each milestone, for your portfolio

How to read the OpenBot source

Read the docs for *why*, the tests for *what*, and the history for *how they got there*. File paths in this doc are from nightly-labs/openbot at commit `0b311bf` (v0.57.0).

1. Clone the full history, not a shallow copy: `git clone https://github.com/nightly-labs/openbot.git`.
2. Read `docs/architecture/structure.md`, `providers.md` and `src/backend/AGENTS.md` before any code. They state the rules the code obeys.
3. Read `docs/glossary.md`. It maps product words (thread, side thread, delivery, lane) to code names.
4. Treat `scripts/*-e2e.ts` and `scripts/provider-fakes/agents/*.ts` as behaviour specs. `stall.ts` and `session-errors.ts` show the failures they guard against.
5. Use `git log -p -- <file>` on the file of your current milestone. Bug fixes show the edge cases you will hit.

6. Follow one message by searching method names: `enqueue` → `scheduleDrain` → `startDelivery` → `"turn/start"` → `handleNotification` → `completeTurn`.
7. Skip `src/renderer`, `src/main` and `apps/` at first. They are product surface, not engine.

Sources

- nightly-labs/openbot on GitHub — repository read at commit `0b311bf`, 2026-11-01